

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled “**MOBMETRICS Mobile and Electronics Item**” submitted to **Indhira Gandhi National Open University** in partial fulfilment of the requirement for the award of the degree of **BACHELOR OF COMPUTER APPLICATIONS (BCA)**, is an original work carried out by Ms. JASEELA T A Enrolment No: **2250341853** under the guidance’s of Ms. **Renju Babu**.

The matter embodied in this project is a genuine work done by the student and has not been submitted whether to this university or any other university/Institute for the fulfilment of the requirement of any course of study.

Signature of the student

Name and Address of the student:

Ms.Jaseela T A

‘Manha’,Santhosh Nagar

Kasaragod, PIN671123.

Signature of the guide

Name,Designation and address of the guide:

Ms. Renju Babu

Software Trainer

Murickanolickal House

Mutholapuram P.O , Ernakulam.

TABLE OF CONTENT

Acknowledgement

Abstract

CHAPTERS	Page no.
1. Introduction	6
1.1. overview	
1.2. Problem statement	
1.3. Solution to the problem	
1.4. Existing system	
1.5. Objectives	
2. Tools and environment used	7
2.1. Programming languages and libraries	
2.2. Tools	
3. Requirement specification	8
3.1. Overall description	
3.1.1. Product perspective	
3.1.2. Product functions	
3.1.3. User characteristics and classes	
3.1.4. Constrains	
3.2. Specific requirements	
3.2.1. Functional requirements	
3.2.2. Non-functional requirements	
3.2.3. Performance requirements	
3.2.4. External interface requirements	
3.2.5. System evolution	
3.3. Use case diagram	
4. Design document	12
4.1 Modularization details	
4.1.1 Front-End modules	
4.1.2 Back-End modules	
4.2 Data Integrity	
4.2.1 Product Data	
4.2.2 User data	
4.2.3 Order data	
4.3 Procedural design	
4.3.1 User registration and login	
4.3.2 product browsing and selection	
4.3.3. Cart	
4.3.4. Payment processing	
4.3.5. Order Confirmation	

4.3.6. Administrative functions	
4.4 User interface design	
4.4.1. User centric design	
4.4.2. Key UI elements and features	
4.4.3. Technology and principles	
4.5 Database design	
5. Program code	20
6. Testing	63
6.1 Unit Testing	
6.2 Integration Testing	
6.2.1 User authentication integration test	
6.2.2 Product Search	
6.2.3 Product selection and ordering	
6.2.4 Admin panel and database integration	
6.3 Code improvement	
6.3.1. Refactoring	
6.3.2. Optimization	
6.3.3. Code reviews	
6.3.4. Version control	
7. Input Output Screens	67
8. Implementation plan	73
9. Limitation of the project	74
10. Future application of the project	75
11. Conclusion	76
12. Bibliography	77
13. Copy of synopsis	78

ACKNOWLEDGEMENT

I want to extend my sincere gratitude to Ms.Renju Babu for their invaluable guidance and support throughout the development of this project. Their expertise and encouragement were instrumental in navigating the challenges and ensuring its successful completion. This project is submitted in partial fulfilment of the Bachelor of Computer Application degree requirements, with sincere gratitude for the assistance I received.

Jaseela T A

Enrolment number: 2250341853

ABSTRACT

The Mobmetrics e-commerce website is a web-based application designed to revolutionize the process of purchasing mobile and electronics item for customers . Addressing the tediousness of traditional methods, which often require physical presence at the shop to check new products and purchase products, this system provides a convenient online platform to access real-time information on latest products, show and availability.

For customers, the system offers features such as online shopping, allowing them to select products, from any location with internet access. This eliminates the need for physical visits and enhances the overall shopping experience. The system also provides updated information on latest products and availability.

For administrators , the system provides tools to efficiently manage products, update products information, monitor orders and delivery , and process transactions.

The application is developed using a combination of front-end and back-end technologies. The front-end, focused on user interface and experience, is built Angular. The back end, responsible for data management and server-side logic, is supported by Nodejs and MySQL.

The system prioritizes user-friendliness, security, and reliability. Future enhancements for the system include the development of mobile applications, multilanguage support, and social media integration.

1. INTRODUCTION

The Mobmetrics e-commerce website is designed to make the shopping experience easier and more efficient. It addresses the common issues of traditional shopping, such as the need for customers to be physically present at market. This system brings the market closer to the customers by providing a website where they can access all the necessary information and shop.

1.1 Overview

This System is a web-based application that provides information about new technologies, show availability. It allows customers to purchase products online and enables admin to manage products, ,stock and availability. The system aims to improve the overall efficiency of shopping both customers and administrators.

1.2 Problem Statement

The traditional shopping process can be tedious and time-consuming. Customers often must physically visit the shop to get information about products, pricing , and product availability. This can lead to uncertainty about product availability and inconvenience for customers.

1.3 Solution to the Problem

The e-commerce System provides a convenient solution by allowing customers to access product information and purchase online. The system provides features such as online shopping capabilities, real-time updates on products availability, and the ability to order product. This eliminates the need for customers to be physically present at the market and simplifies the shopping.

1.4 Existing System

The existing system requires customers to visit the market physically to inquire about product details and shop. This process is often time-consuming and inconvenient.

1.5 Objectives

The main objectives of the Mobmetrics e-commerce System are to:

- Provide customers with online shopping facilities
- Offer real-time information on product listings, pricing, and availability
- Enable admin to efficiently manage products stock and orders
- Improve the overall customer experience by providing a user-friendly and convenient shopping process

2.TOOLS AND ENVIRONMENT USED

2.1 Programming languages and libraries used:

- Front-end: Angular
- Back-end: NodeJs , MySQL

2.2 Tools:

- Platform: Windows 11 / Ubuntu
- Tools:code editor (VS Code), browser & debugger (Chromium-based)

3. REQUIREMENT SPECIFICATION

3.1 Overall Description

3.1.1 Product Perspective:

- This System is designed as a software application suite intended to enhance the operational efficiency of Mobile and electronics sales and management.
- It serves as a tool for administrators to maintain and update products, stock availability, and pricing.
- The system also provides a user-friendly interface to manage shopping, check availability, and process transactions.
- Most importantly, it offers the public a convenient platform to browse products listings, check pricing, and purchase products online, reducing the need for physical visits to the market.
- The system aims to integrate various functionalities such as product information display, ordering, payment processing, and database management into a single, cohesive platform.

3.1.2 Product Functions:

- The system's core function is to provide real-time information about products, including stock, price,
- For customers, the system facilitates online shopping, allowing them to select products, and complete the purchase through online payment gateways.
- Administrators can manage stock inventory, update stock, modify availability, and oversee the system's overall operation.
- Admin can utilize the system to view order details, manage user queries, and handle product cancellations or refunds.
- The system also includes a refund mechanism, enabling users to cancel order and claim a partial refund under specific conditions (e.g., 25% refund if cancelled before shipping).

3.1.3 User Characteristics and Classes:

- The system caters to three primary user classes: customers, seller, and administrators.
- Customers are expected to have basic computer literacy and familiarity with internet browsing to navigate the online shopping platform.
- seller require a moderate level of system knowledge to manage product order.
- Administrators need advanced system proficiency to manage the system's database, update product category information, and perform maintenance tasks.
- To ensure ease of use, the system is designed with a user-friendly interface, accompanied by user manuals, online help resources, and installation guides.

3.1.4 Constraints:

- The system is constrained by the requirement to securely store user information in a database that is accessible to the Online System.
- Security considerations necessitate that the System's security measures are compatible with internet application security standards.
- Accessibility is a constraint, as users are expected to access the system via computers with internet browsing capabilities and a stable internet connection.
- User authentication is a key constraint, with the system requiring users to have valid usernames and passwords to gain access.

3.2 Specific Requirements:

3.2.1 Functional Requirements:

- Logon Capabilities: The system must provide secure logon functionality for all user classes, ensuring that only authorized users can access the system.
- Alerts: The system should be capable of generating alerts to notify administrators of any system-related issues or potential problems.

3.2.2 Non-Functional Requirements:

- **Usability:**
 - The system should be accessible through standard internet browsers, utilizing HTML or similar web technologies for the user interface.
 - The user interface should be intuitive and require minimal training, leveraging users' familiarity with web browsers.
 - The system should be self-explanatory, guiding users through various functions and processes without extensive external assistance.
- **Reliability:**
 - The system's reliability is critical to ensure data integrity and prevent disruptions to ticket sales and management.
 - **Availability:** The system is expected to maintain a high level of availability, aiming for 24/7 operation throughout the year.
 - **Mean Time to Repair (MTTR):** In the event of a system failure, the system should be recoverable within a short timeframe (e.g., within an hour).
 - **Accuracy:** The accuracy of the system's data and operations is dependent on the accuracy of user inputs and administrative actions.
 - **Probable Bugs:** Potential sources of errors or bugs include issues with payment gateway processing and instability in user internet connectivity.

3.2.3 Performance Requirements:

- **Response Time:** The system should exhibit fast response times, such as loading the homepage with product listings in under 2 seconds.
- **Real-time updates:** like refreshing seat availability after a transaction, are crucial.
- **User Action:** The system should generally respond to user actions, like payment completion, within a few seconds, with allowances for longer processing times for bulk order.
- **Administrator Response:** The system should prioritize quick response times for administrator functions to ensure efficient system management.
- **Throughput:** The system's throughput, measured by the number of orders, is directly related to sales volume and user activity.
- **Resource Utilization:** The system's resource allocation should be dynamic, adapting to user demands and the volume of product order requests.

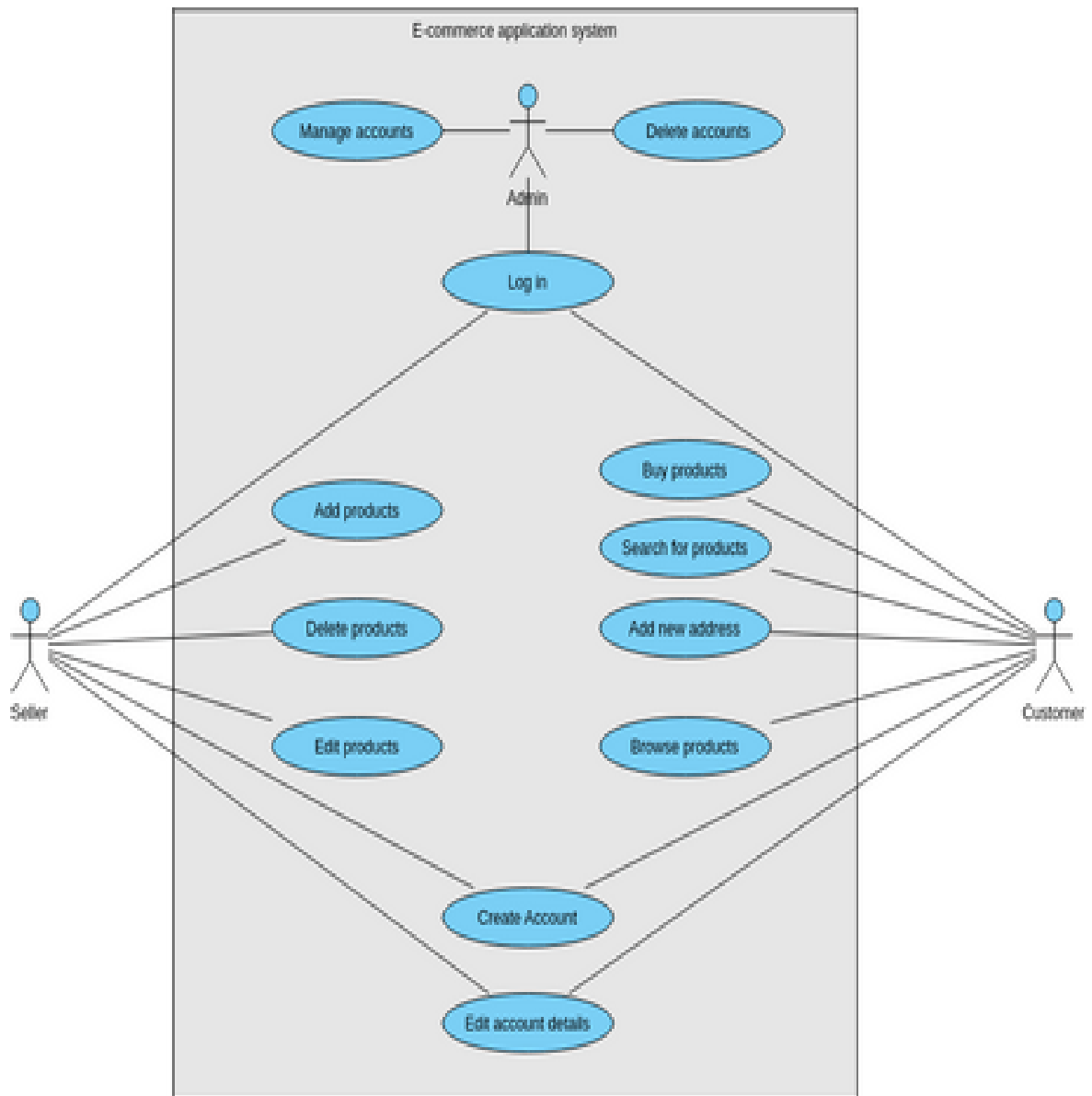
3.2.4 External Interface Requirements:

- **Internet Protocols:** The system must adhere to standard internet communication protocols, such as TCP/IP, to ensure compatibility and interoperability.
- **Cancellation Flexibility:** The system should provide users with the ability to cancel order within a specified timeframe (e.g., prior to the shipping).
- **Refund :** The system should also include a mechanism for processing refunds, adhering to defined policies and procedures.
- **Maintenance:** Regular maintenance of the system is necessary to ensure optimal performance, with maintenance activities scheduled at consistent intervals (e.g., weekly).
- **Standards:** The system's development should follow industry-standard coding practices and naming conventions to promote code readability, maintainability, and collaboration.
- **Design Constraints:**
 - The system's front-end development is constrained to using web technologies to create the user interface and interactive elements.
 - The back-end development is limited to using NodeJS server-side logic and MySQL for database management.

3.2.5 System Evolution:

- The system is designed to be scalable and adaptable, with potential future enhancements including partnerships with additional business entity.
- Performance improvements, such as increasing database access speed, are also considered for future development.

3.3 USE CASE DIAGRAM



4. DESIGN DOCUMENT

4.1 Modularization details

4.1.1 Front-End Modules:

- **Homepage Module:** Displays the products and provides an overview of the system.
- **Product Listing Module:** Shows details of products, including Name, specification, and price.
- **Single product view:** Enables users to view single product.
- **Search Module:** Allows users to search for products, related information.
- **Buy product Module:** Handles the process of purchasing product.
- **Login Module:** Provides user authentication and access control.
- **Navigation Module:** Consists of navigation bars for easy access to different modules

4.1.2 Back-End Modules

- **Database Management Module:** Manages the storage and retrieval of data related to movies, users, theaters, and bookings.
- **Server-Side Logic Module:** Processes user requests, handles transactions, and interacts with the database.
- **Payment Gateway Integration Module:** Securely processes online payments for ticket purchases.
- **User Authentication Module:** Verifies user credentials and manages user sessions.
- **Admin Module:** Manages order request, updates availability, and handles other administrative tasks.

4.2 Data integrity

4.2.1 Movie Data:

- **Data Elements:**
Product name , Specification, price,
- **Integrity Considerations:**
 - **Accuracy:** Ensuring that product information is correct and up-to date. This involves regular updates by administrative staff.
 - **Consistency:** Maintaining consistent formatting and terminology for product details.
 - **Completeness:** All necessary details (title, price, etc.) are present for each product.

4.2.2 Product Availability Data:

- **Data Elements:** Stock, availability status (available).
- **Integrity Considerations:**
 - Accuracy: Reflecting the correct availability of product in real-time.

4.2.3 User Data:

- **Data Elements:** Usernames, passwords, contact information.
- **Integrity Considerations:**
 - Accuracy: Ensuring user information is correct.
 - Validity: Usernames and passwords must adhere to security policies.
 - Confidentiality: Protecting user data from unauthorized access.

4.2.4 Order Data:

- **Data Elements:** order details, payment information.
- **Integrity Considerations:**
 - Accuracy: Ensuring order details are correct and match user selections.
 - Completeness: All necessary order information is recorded.
 - Validity: ordered product must be valid.

4.3 Procedural design

4.3.1 User Registration and Login:

- Users initiate the process by accessing the Login Module.
- New users provide necessary details (e.g., username, password, contact information) to register.
- Registered users enter their credentials (username and password) to log in to the system.
- The system verifies the credentials and grants access to the user's account.

4.3.2 Movie Browsing and Selection:

- Users can browse products through the Homepage Module or the product Listing Module.
- The system displays product information, including name, specification, and price.
- Users may use the Search Module to find specific product or category.
- Users can view specific product using the single product Module.

4.3.3 Product ordering:

- Users select a product, and the number of items.
- The system calculates the total cost.

4.3.4 Payment Processing:

- Users proceed to the cart and Module to make the payment.
- The system integrates with a Payment Gateway Module to handle online transactions.
- Users enter their payment details.
- The system securely processes the payment.

4.3.5 Order Confirmation and Delivery:

- The system validates the payment.
- The system sends a digital copy of the ticket to the user (e.g., via email).

4.3.6 Administrative Functions:

- Administrators log in to the system using their credentials.
- Administrators can manage seller, category, and update product availability.
- The system updates the database with any changes made by the administrators.
- Administrators can monitor order and generate reports.

4.4 User interface design**4.4.1 User-Centric Design:**

- The UI is designed to be user-friendly, focusing on ease of navigation and intuitive interactions.
- The design prioritizes usability and efficiency to ensure a positive user experience.

4.4.2 Key UI Elements and Features:

- Homepage:
 - Displays the products.
- Navigation Bars:
 - Provide clear navigation to different modules (e.g., Login, shop cart etc).

- Product Listings:
 - Showcase product details, including titles, price , and image.
 - Users can typically click on specific product to view additional information.
- Single product Section:
 - Allows users to see specific product .
 - viewing is accessible without requiring users to log in.
- Search Box:
 - Enables users to search for products, category, and other relevant information.
- Product ordering System:
 - Facilitates the selection of product ,
 - Displays selected products in cart.
- Payment Gateways:
 - Securely handles online payment transactions

4.4.3 Technology and Design Principles:

- The front-end is developed using Angular
- HTML provides the structure and design of web browser documents
- CSS enhances the visual presentation and styling of the website
- The UI design aims to integrate interface design and user experience (UX) design principles
- UX design focuses on creating a positive user experience through user product interaction

5.DATABASE DESIGN

User

Field Name	Data Type	Constraint	Description
User_id	Int	Primary Key	User ID
Email	Varchar(20)	Not Null	Login Email
Password	Varchar(20)	Not Null	Password
First Name	Varchar(20)	Not Null	First Name
Last Name	Varchar(20)	Not Null	Last Name
Address	Varchar(100)	Not Null	Address
Pin	Int	Not Null	Pin Code
Phone	Varchar(20)	Not Null	Phone Number
Role	Varchar(20)	Not Null	User Role

Product

Field Name	Data Type	Constraint	Description
Product_id	Int	Primary Key	Product ID
Product_Name	Varchar(20)	Not Null	Product Name
Product_Desc	Varchar(50)	Not Null	Product Description
Image1	Varchar(20)	Not Null	Product Image1
Image2	Varchar(20)	Not Null	Product Image2
Image3	Varchar(100)	Not Null	Product image3
Price	Int	Not Null	Product Price
Seller_ID	Int	Foreign Key	Seller ID
Category_ID	Int	Foreign Key	Category ID
Stock	Int	Not Null	Stock

Categories

Field Name	Data Type	Constraint	Description
Category_ID	Int	Primary Key	Product ID
Name	Varchar(20)	Not Null	Category Name
Description	Varchar(50)	Not Null	Description

Seller

Field Name	Data Type	Constraint	Description
Seller_ID	Int	Primary Key	Seller ID
User_ID	Int	Foreign Key	User ID
Store_name	Varchar(50)	Not Null	Name of the Seller
Desc	Varchar(50)	Not Null	Store Description
Address	Varchar(100)	Not Null	Store Address

Order

Field Name	Data Type	Constraint	Description
Order_ID	Int	Primary Key	Order ID
User_ID	Int	Foreign Key	User ID
Total Amount	Int	Not Null	Tota Amount
Date	Date	Not Null	Date
Status	Varchar(50)	Not Null	Status

Order Items

Field Name	Data Type	Constraint	Description
Order_Item_ID	Int	Primary Key	Order item ID
Order_ID	Int	Foreign Key	Order ID
Product_ID	Int	Foreign Key	Products ID

Quantity	Int	Not Null	No.of Products
Price	Decimal(10,2)	Not Null	Price of the Product

Payment

Field Name	Data Type	Constraint	Description
Payment_ID	Int	Primary Key	Payment ID
Order_ID	Int	Foreign Key	Order ID
Method	Varchar(20)	Not Null	Payment Method
Status	Varchar(20)	Not Null	Payment Status
Amount	Decimal(10,2)	Not Null	Order Amount
Time	Timestamp	Not Null	Date and Time

Cart

Field Name	Data Type	Constraint	Description
Cart_ID	Int	Primary Key	Cart ID
User_ID	Int	Foreign Key	User ID
Product_ID	Int	Foreign Key	Product Reference
Quantity	Int	Not Null	Product Quantity

Services

Field Name	Data Type	Constraint	Description
Service_ID	Int	Primary Key	Service ID
Seller_ID	Int	Foreign Key	Seller Reference
User_ID	Int	Foreign Key	User Reference
Product_ID	Int	Foreign Key	Product Reference
Desc	Varchar(20)	Not Null	Desc of the Service
Amount	Decimal(10,2)	Not Null	Service amount

Duration	Int	Not Null	Service Duration
Date	Date	Not Null	Date and time

Review

Field Name	Data Type	Constraint	Description
Review_id	Int	Primary Key	Review ID
User_ID	Int	Foreign Key	User Reference
Product_ID	Int	Foreign Key	Product Reference
Service_ID	Int	Foreign Key	Service Reference
Review	Varchar(100)	Not Null	Detailed Review
Rating	Int	Not Null	Rating
Date	Date	Not Null	Date and time

5. PROGRAM CODE

Guesmaster.component.html

```
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Responsive Bootstrap4 Shop Template, Created by
Imran Hossain from https://imransdesign.com/">
  <!-- title -->
  <title>Mobmetrics</title>

  <!-- favicon -->

  <link rel="shortcut icon" type="image/png"
href="/assets/guest/assets/img/favicon.png">

  <!-- google font -->

  <link href="https://fonts.googleapis.com/css?family=Open+Sans:300,400,700"
rel="stylesheet">

  <link href="https://fonts.googleapis.com/css?family=Poppins:400,700&display=swap"
rel="stylesheet">

  <!-- fontawesome -->

  <link rel="stylesheet" href="/assets/guest/assets/css/all.min.css">

  <!-- bootstrap -->

  <link rel="stylesheet" href="/assets/guest/assets/bootstrap/css/bootstrap.min.css">

  <!-- owl carousel -->

  <link rel="stylesheet" href="/assets/guest/assets/css/owl.carousel.css">

  <!-- magnific popup -->

  <link rel="stylesheet" href="/assets/guest/assets/css/magnific-popup.css">

  <!-- animate css -->

  <link rel="stylesheet" href="/assets/guest/assets/css/animate.css">

  <!-- mean menu css -->

  <link rel="stylesheet" href="/assets/guest/assets/css/meanmenu.min.css">

  <!-- main style -->

  <link rel="stylesheet" href="/assets/guest/assets/css/main.css">

  <!-- responsive -->

  <link rel="stylesheet" href="/assets/guest/assets/css/responsive.css">
</head>
```

```

<!-- header -->
<div class="top-header-area" id="sticker">
  <div class="container">
    <div class="row">
      <div class="col-lg-12 col-sm-12 text-center">
        <div class="main-menu-wrap">
          <!-- logo -->
          <div class="site-logo">
            <a href="/guestmaster/guestindex">
              
            </a>
          </div>
          <!-- logo -->
        <!-- menu start -->
        <nav class="main-menu">
          <ul>
            <li class="current-list-item"><a
href="/guestmaster/guestindex">Home</a>
            </li>
            <li><a href="about.html">About</a></li>
            <li><a href="#">Pages</a>
            <ul class="sub-menu">
              <li><a href="404.html">404 page</a></li>
              <li><a href="about.html">About</a></li>
              <li><a href="cart.html">Cart</a></li>
              <li><a href="checkout.html">Check Out</a></li>
              <li><a href="contact.html">Contact</a></li>
              <li><a href="news.html">News</a></li>
              <li><a href="shop.html">Shop</a></li>
            </ul>
          </li>

```

```

<li><a href="news.html">News</a>
  <ul class="sub-menu">
    <li><a href="news.html">News</a></li>
    <li><a href="single-news.html">Single News</a></li>
  </ul>
</li>
<li><a href="contact.html">Contact</a></li>
<li><a href="shop.html">Shop</a>
  <ul class="sub-menu">
    <li><a href="shop.html">Shop</a></li>
    <li><a href="checkout.html">Check Out</a></li>
    <li><a href="single-product.html">Single Product</a></li>
    <li><a href="cart.html">Cart</a></li>
  </ul>
</li>
<li><a href="login">Login</a></li>
<li>
  <div class="header-icons">
    <a class="shopping-cart" href="cart.html"><i class="fas fa-
shopping-cart"></i></a>
    <a class="mobile-hide search-bar-icon" href="#"><i class="fas
fa-search"></i></a>
  </div>
</li>
</ul>
</nav>
<a class="mobile-show search-bar-icon" href="#"><i class="fas fa-
search"></i></a>
<div class="mobile-menu"></div>
<!-- menu end -->
</div>
</div>
</div>
</div>

```

```

</div>
<!-- end header -->

<!-- search area -->
<div class="search-area">
  <div class="container">
    <div class="row">
      <div class="col-lg-12">
        <span class="close-btn"><i class="fas fa-window-close"></i></span>
        <div class="search-bar">
          <div class="search-bar-tablecell">
            <h3>Search For:</h3>
            <input type="text" placeholder="Keywords">
            <button type="submit">Search <i class="fas fa-search"></i></button>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
<!-- end search area -->

<router-outlet></router-outlet>

<!-- footer -->
<div class="footer-area">
  <div class="container">
    <div class="row">
      <div class="col-lg-3 col-md-6">
        <div class="footer-box about-widget">
          <h2 class="widget-title">About us</h2>
          <p>Start selling with us and reach millions of our customers,Place your
products in front of high-intent shoppers and convert them with ease</p>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

</div>
<div class="col-lg-3 col-md-6">
  <div class="footer-box get-in-touch">
    <h2 class="widget-title">Get in Touch</h2>
    <ul>
      <li>34/8, East Hukupara, Gifirtok, Sadan.</li>
      <li></li>
      <li>+00 111 222 3333</li>
    </ul>
  </div>
</div>
<div class="col-lg-3 col-md-6">
  <div class="footer-box pages">
    <h2 class="widget-title">Pages</h2>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="services.html">Shop</a></li>
      <li><a href="news.html">News</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </div>
</div>
<div class="col-lg-3 col-md-6">
  <div class="footer-box subscribe">
    <h2 class="widget-title">Subscribe</h2>
    <p>Subscribe to our mailing list to get the latest updates.</p>
    <form action="index.html">
      <input type="email" placeholder="Email">
      <button type="submit"><i class="fas fa-paper-plane"></i></button>
    </form>
  </div>

```



```

        </div>
    </div>
</div>
</div>
<!-- end footer -->

<!-- copyright -->
<div class="copyright">
    <div class="container">
        <div class="row">
            <div class="col-lg-6 col-md-12">

                </div>
                <div class="col-lg-6 text-right col-md-12">
                    <div class="social-icons">
                        <ul>
                            <li><a href="#" target="_blank"><i class="fab fa-facebook-
f"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
twitter"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
instagram"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
linkedin"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
dribbble"></i></a></li>
                        </ul>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
<!-- end copyright -->

```

```

<!-- jquery -->
<script src="/assets/guest/assets/js/jquery-1.11.3.min.js"></script>
<!-- bootstrap -->
<script src="/assets/guest/assets/bootstrap/js/bootstrap.min.js"></script>
<!-- count down -->
<script src="/assets/guest/assets/js/jquery.countdown.js"></script>
<!-- isotope -->
<script src="/assets/guest/assets/js/jquery.isotope-3.0.6.min.js"></script>
<!-- waypoints -->
<script src="/assets/guest/assets/js/waypoints.js"></script>
<!-- owl carousel -->
<script src="/assets/guest/assets/js/owl.carousel.min.js"></script>
<!-- magnific popup -->
<script src="/assets/guest/assets/js/jquery.magnific-popup.min.js"></script>
<!-- mean menu -->
<script src="/assets/guest/assets/js/jquery.meanmenu.min.js"></script>
<!-- sticker js -->
<script src="/assets/guest/assets/js/sticker.js"></script>
<!-- main js -->
<script src="/assets/guest/assets/js/main.js"></script>

```

Guestindex.component.html

```

<!-- hero area -->
<div class="hero-area hero-bg">
  <div class="container">
    <div class="row">
      <div class="col-lg-9 offset-lg-2 text-center">
        <div class="hero-text">
          <div class="hero-text-tablecell">
            <p class="subtitle">Mobmetrics</p>
            <h1>Mobiles and Electronics</h1>

```

```

        <div class="hero-btns">
            <a href="/guestmaster/shop" class="boxed-btn">Products</a>
            <a href="contact.html" class="bordered-btn">Contact Us</a>
        </div>
    </div>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
<!-- end hero area -->

<!-- features list section -->
<div class="list-section pt-80 pb-80">
    <div class="container">

        <div class="row">
            <div class="col-lg-4 col-md-6 mb-4 mb-lg-0">
                <div class="list-box d-flex align-items-center">
                    <div class="list-icon">
                        <i class="fas fa-shipping-fast"></i>
                    </div>
                    <div class="content">
                        <h3>Free Shipping</h3>
                        <p>When order over $75</p>
                    </div>
                </div>
            </div>
            <div class="col-lg-4 col-md-6 mb-4 mb-lg-0">
                <div class="list-box d-flex align-items-center">
                    <div class="list-icon">
                        <i class="fas fa-phone-volume"></i>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

    </div>
    <div class="content">
        <h3>24/7 Support</h3>
        <p>Get support all day</p>
    </div>
</div>
</div>
<div class="col-lg-4 col-md-6">
    <div class="list-box d-flex justify-content-start align-items-center">
        <div class="list-icon">
            <i class="fas fa-sync"></i>
        </div>
        <div class="content">
            <h3>Refund</h3>
            <p>Get refund within 3 days!</p>
        </div>
    </div>
</div>
</div>
</div>
</div>
</div>
<!-- end features list section -->

<!-- product section -->
<div class="product-section mt-150 mb-150">
    <div class="container">
        <div class="row">
            <div class="col-lg-8 offset-lg-2 text-center">
                <div class="section-title">
                    <h3><span class="orange-text">Our</span> Products</h3>
                    <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Aliquid, fuga
                    quas itaque eveniet beatae optio.</p>

```

```

    </div>
  </div>
</div>

<div class="row">
  <div class="col-lg-4 col-md-6 text-center">
    <div class="single-product-item">
      <div class="product-image">
        <a href="single-product.html"></a>
      </div>
      <h3>iphone10</h3>
      <p class="product-price"><span></span> Rs.45000 </p>
      <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
  </div>
  <div class="col-lg-4 col-md-6 text-center">
    <div class="single-product-item">
      <div class="product-image">
        <a href="single-product.html"></a>
      </div>
      <h3>iphone11</h3>
      <p class="product-price"><span>Per Kg</span> Rs.42000 </p>
      <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
  </div>
  <div class="col-lg-4 col-md-6 offset-md-3 offset-lg-0 text-center">
    <div class="single-product-item">
      <div class="product-image">
        <a href="single-product.html"></a>

```

```

        </div>
        <h3>iphone13</h3>
        <p class="product-price"><span>Per Kg</span> R.44990 </p>
        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
</div>
</div>
</div>
</div>
<!-- end product section -->

<!-- cart banner section -->
<section class="cart-banner pt-100 pb-100">
    <div class="container">
        <div class="row clearfix">
            <!--Image Column-->
            <div class="image-column col-lg-6">
                <div class="image">
                    <div class="price-box">
                        <div class="inner-price">
                            <span class="price">
                                <strong>30%</strong> <br> off per kg
                            </span>
                        </div>
                    </div>
                </div>
            </div>
            <!--Content Column-->
            <div class="content-column col-lg-6">
                <h3><span class="orange-text">Deal</span> of the month</h3>
                <h4></h4>
            </div>
        </div>
    </div>

```

```

<!--Countdown Timer-->

<div class="time-counter"><div class="time-countdown clearfix" data-
countdown="2020/2/01"><div class="counter-column"><div class="inner"><span
class="count">00</span>Days</div></div> <div class="counter-column"><div
class="inner"><span class="count">00</span>Hours</div></div> <div class="counter-
column"><div class="inner"><span class="count">00</span>Mins</div></div> <div
class="counter-column"><div class="inner"><span
class="count">00</span>Secs</div></div></div></div>

<a href="cart.html" class="cart-btn mt-3"><i class="fas fa-shopping-
cart"></i> Add to Cart</a>

```

```

</div>
</div>
</div>
</section>
<!-- end cart banner section -->

```

```

<!-- testimonail-section -->
<div class="testimonial-section mt-150 mb-150">
<div class="container">
<div class="row">
<div class="col-lg-10 offset-lg-1 text-center">
<div class="testimonial-sliders">
<div class="single-testimonial-slider">
<div class="client-avater">

</div>
<div class="client-meta">
<h3>Saira Hakim <span>Local shop owner</span></h3>
<p class="testimonial-body">

</p>
<div class="last-icon">
<i class="fas fa-quote-right"></i>
</div>
</div>

```

```

</div>
<div class="single-testimonial-slider">
  <div class="client-avater">
    
  </div>
  <div class="client-meta">
    <h3>David Niph <span>Local shop owner</span></h3>
    <p class="testimonial-body">
    </p>
    <div class="last-icon">
      <i class="fas fa-quote-right"></i>
    </div>
  </div>
</div>
<div class="single-testimonial-slider">
  <div class="client-avater">
    
  </div>
  <div class="client-meta">
    <h3>Jacob Sikim <span>Local shop owner</span></h3>
    <p class="testimonial-body">
    </p>
    <div class="last-icon">
      <i class="fas fa-quote-right"></i>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>

```



```

<!-- end testimonail-section -->

<!-- advertisement section -->
<div class="abt-section mb-150">
  <div class="container">
    <div class="row">
      <div class="col-lg-6 col-md-12">
        <div class="abt-bg">
          <a href="https://www.youtube.com/watch?v=DBLIFWYcIGQ"
class="video-play-btn popup-youtube"><i class="fas fa-play"></i></a>
        </div>
      </div>
      <div class="col-lg-6 col-md-12">
        <div class="abt-text">
          <p class="top-sub">Since Year 1999</p>
          <h2>We are <span class="orange-text">Fruitkha</span></h2>
          <a href="about.html" class="boxed-btn mt-4">know more</a>
        </div>
      </div>
    </div>
  </div>
<!-- end advertisement section -->

<!-- shop banner -->
<section class="shop-banner">
  <div class="container">
    <h3>December sale is on! <br> with big <span class="orange-
text">Discount...</span></h3>
    <div class="sale-percent"><span>Sale! <br> Upto</span>50%
<span>off</span></div>
    <a href="shop.html" class="cart-btn btn-lg">Shop Now</a>
  </div>
</section>

```

```

<!-- end shop banner -->

<!-- latest news -->
<div class="latest-news pt-150 pb-150">
  <div class="container">

    <div class="row">
      <div class="col-lg-8 offset-lg-2 text-center">
        <div class="section-title">
          <h3><span class="orange-text">Our</span> News</h3>
        </div>
      </div>
    </div>

    <div class="row">
      <div class="col-lg-4 col-md-6">
        <div class="single-latest-news">
          <a href="single-news.html"><div class="latest-news-bg news-bg-
1"></div></a>
          <div class="news-text-box">
            <p class="blog-meta">
              <span class="author"><i class="fas fa-user"></i> Admin</span>
              <span class="date"><i class="fas fa-calendar"></i> 27 December,
2019</span>
            </p>
            <a href="single-news.html" class="read-more-btn">read more <i
class="fas fa-angle-right"></i></a>
          </div>
        </div>
      </div>
      <div class="col-lg-4 col-md-6">
        <div class="single-latest-news">
          <a href="single-news.html"><div class="latest-news-bg news-bg-
2"></div></a>

```

```

<div class="news-text-box">
  <h3><a href="single-news.html">A man's worth has its season, like
tomato.</a></h3>
  <p class="blog-meta">
    <span class="author"><i class="fas fa-user"></i> Admin</span>
    <span class="date"><i class="fas fa-calendar"></i> 27 December,
2019</span>
  </p>
  <p class="excerpt">Vivamus lacus enim, pulvinar vel nulla sed,
scelerisque rhoncus nisi. Praesent vitae mattis nunc, egestas viverra eros.</p>
  <a href="single-news.html" class="read-more-btn">read more <i
class="fas fa-angle-right"></i></a>
</div>
</div>
</div>
<div class="col-lg-4 col-md-6 offset-md-3 offset-lg-0">
  <div class="single-latest-news">
    <a href="single-news.html"><div class="latest-news-bg news-bg-
3"></div></a>
    <div class="news-text-box">
      <h3><a href="single-news.html">Good thoughts bear good fresh juicy
fruit.</a></h3>
      <p class="blog-meta">
        <span class="author"><i class="fas fa-user"></i> Admin</span>
        <span class="date"><i class="fas fa-calendar"></i> 27 December,
2019</span>
      </p>
      <a href="single-news.html" class="read-more-btn">read more <i
class="fas fa-angle-right"></i></a>
    </div>
  </div>
</div>
</div>
<div class="row">

```

```

    <div class="col-lg-12 text-center">
        <a href="news.html" class="boxed-btn">More News</a>
    </div>
</div>
</div>
</div>
</div>
<!-- end latest news -->

```

Login.component.ts

```

import { Component } from "@angular/core";
import { AuthService } from "../services/auth.service";

@Component({
  selector: "app-login",
  templateUrl: "../login.component.html",
})
export class LoginComponent {
  credentials = { email: "", password: "" };

  constructor(private authService: AuthService) {}

  login() {
    this.authService.login(this.credentials).subscribe((res) => {
      localStorage.setItem("token", res.token);
      alert("Login successful!");
    });
  }
}

```

Login.component.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Login V2</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/png" href="/assets/login/images/icons/favicon.ico"/>
  <link rel="stylesheet" type="text/css"
href="/assets/login/vendor/bootstrap/css/bootstrap.min.css">
  <link rel="stylesheet" type="text/css" href="/assets/login/fonts/font-awesome-
4.7.0/css/font-awesome.min.css">
<!--
  <link rel="stylesheet" type="text/css" href="/assets/login/fonts/iconic/css/material-
design-iconic-font.min.css">

```

```

<link rel="stylesheet" type="text/css" href="/assets/login/vendor/animate/animate.css">

<link rel="stylesheet" type="text/css" href="/assets/login/vendor/css-
hamburgers/hamburgers.min.css">
<link rel="stylesheet" type="text/css"
href="/assets/login/vendor/animation/css/animation.min.css">
<link rel="stylesheet" type="text/css"
href="/assets/login/vendor/select2/select2.min.css">
<link rel="stylesheet" type="text/css"
href="/assets/login/vendor/daterangepicker/daterangepicker.css">
<link rel="stylesheet" type="text/css" href="/assets/login/css/util.css">
<link rel="stylesheet" type="text/css" href="/assets/login/css/main.css">
</head>
<body>

<div class="limiter">
  <div class="container-login100">
    <div class="wrap-login100">
      <form (ngSubmit)="submit()" [formGroup]="loginform" class="login100-
form">
        <span class="login100-form-title p-b-26">
          Login
        </span>

        <div class="wrap-input100 " >
          <input class="input100" type="text" name="email"
formControlName="email">

        </div>

        <div class="wrap-input100 " >
          <span class="btn-show-pass">
            <i class="zmdi zmdi-eye"></i>
          </span>
          <input class="input100" type="password" name="pass"
formControlName="password">

        </div>

        <button type="submit">
          Login
        </button>
        <div class="text-center p-t-115">
          <span class="txt1">
            Don't have an account?
          </span>
          <a class="txt2" href="customerregistration">
            Sign Up
          </a>
        </div>
      </form>
    </div>
  </div>

```

```

        </form>

    </div>
</div>

<div id="dropDownSelect1"></div>

<script src="/assets/login/vendor/jquery/jquery-3.2.1.min.js"></script>
<script src="/assets/login/vendor/animation/js/animation.min.js"></script>
<script src="/assets/login/vendor/bootstrap/js/popper.js"></script>
<script src="/assets/login/vendor/bootstrap/js/bootstrap.min.js"></script>
<script src="/assets/login/vendor/select2/select2.min.js"></script>
<script src="/assets/login/vendor/daterangepicker/moment.min.js"></script>
<script src="/assets/login/vendor/daterangepicker/daterangepicker.js"></script>
<script src="/assets/login/vendor/countdown/countdown.js"></script>
<script src="/assets/login/js/main.js"></script>

</body>
</html>

```

Customerregistration.component.js

```

const express = require("express");
const sql = require("mssql");
const bcrypt = require("bcrypt");
const cors = require("cors");

const app = express();
app.use(express.json());
app.use(cors());

const config = {
  user: "your_db_user",
  password: "your_db_password",
  server: "localhost",
  database: "your_db_name",
  options: {
    encrypt: true,
    trustServerCertificate: true,
  },
};

sql.connect(config, (err) => {
  if (err) console.log(err);
  console.log("Connected to MSSQL");
});

```

```
// Register customer
app.post("/register", async (req, res) => {
  const { name, email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);

  const request = new sql.Request();
  request.input("name", sql.VarChar, name);
  request.input("email", sql.VarChar, email);
  request.input("password", sql.VarChar, hashedPassword);

  request.query(
    "INSERT INTO customers (name, email, password) VALUES (@name, @email, @password)",
    (err) => {
      if (err) return res.status(500).json({ error: err.message });
      res.json({ message: "Customer registered successfully!" });
    }
  );
});

app.listen(3000, () => console.log("Server running on port 3000"));
```

Customerregistration.component.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Simple Reistration Form</title>
</head>
<body>
  <form>
    <div class="container">
      <h1>Sign Up</h1>
      <p>Please fill in this form to create an account.</p>
```

<label for="email">First Name</label>

<input type="text" name="firstname" placeholder="Enter First Name" required>

<label for="email">Last Name</label>

<input type="text" name="lastname" placeholder="Enter Last Name" required>

<label for="email">Address</label>

<input type="text" name="address" placeholder="Enter Address" required>

<label for="email">PIN </label>

<input type="text" name="pin" placeholder="Enter Pin Code" required>

<label for="email">Phone Number</label>

<select name="phoneCode" required>

<option selected hidden value="">+91</option>

<option value="91">+91</option>

<option value="66">+91</option>

<option value="66">+90</option>

<option value="66">+66</option>

</select>

<input type="phone" name="phone" required>

<label for="email">Username</label>

<input type="text" name="username" placeholder="Enter Username" required>

<label for="email">Email</label>

<input type="text" placeholder="Enter Email" name="email" required>

<label for="psw">Password</label>

<input type="password" placeholder="Enter Password" name="psw" required>

<label for="psw">Confirm Password</label>

<input type="password" placeholder="Enter Password" name="psw" required>

<p>By creating an account you agree to our Terms & Privacy.</p>

<div class="clearfix">

<button type="submit" class="btn">Sign Up</button>


```

    </div>
  </div>
</form>
<div class="youtubeBtn">
  <a target="_blank" href="https://www.youtube.com/watch?v=7k8kKRQa9jY">
    <span>Watch on YouTube</span>
    <i class="fab fa-youtube"></i>
  </a>
</div>
</body>
</html>

```

Customerregistration.component.ts

```

import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";

@Injectable({
  providedIn: "root",
})
export class AuthService {
  private apiUrl = "http://localhost:3000/register";

  constructor(private http: HttpClient) {}

  register(userData: any) {
    return this.http.post(this.apiUrl, userData);
  }
}

```

Shop.ts

```

import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";

@Injectable({
  providedIn: "root"
})
export class ProductService {
  private apiUrl = "http://localhost:3000/products";

  constructor(private http: HttpClient) {}

  getProducts() {

```

```

    return this.http.get(this.apiUrl);
  }
}

```

Shop.js

```

const express = require("express");
const mysql = require("mysql");
const cors = require("cors");

const app = express();
app.use(cors());

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "products_db",
});

db.connect(err => {
  if (err) throw err;
  console.log("Connected to MySQL");
});

app.get("/products", (req, res) => {
  db.query("SELECT * FROM products", (err, results) => {
    if (err) throw err;
    res.json(results);
  });
});

app.listen(3000, () => console.log("Server running on port 3000"));

```

Shop.html

```

<!-- breadcrumb-section -->
<div class="breadcrumb-section breadcrumb-bg">
  <div class="container">
    <div class="row">
      <div class="col-lg-8 offset-lg-2 text-center">
        <div class="breadcrumb-text">
          <h1><p>Mobmetrics</p></h1>
          <!-- <h1>Shop</h1> -->
        </div>
      </div>
    </div>
  </div>
</div>
<!-- end breadcrumb section -->

```

```

<!-- products -->
<div class="product-section mt-150 mb-150">
  <div class="container">

    <div class="row">
      <div class="col-md-12">
        <div class="product-filters">
          <ul>
            <li class="active" data-filter="*">All</li>
            <li data-filter=".mobile">Mobiles</li>
            <li data-filter=".wearable">Smart Watches</li>
            <li data-filter=".Accessories">Mobile Accesories</li>
          </ul>
        </div>
      </div>
    </div>

    <div class="row product-lists">
      <div class="col-lg-4 col-md-6 text-center strawberry">
        <div class="single-product-item">
          <div class="product-image">
            <a href="/guestmaster/singleproduct"></a>
          </div>
          <h3>Apple iPhone 15 (256GB, Blue)</h3>
          <p class="product-price"><span></span> Rs.72400 </p>
          <a href="cart" class="cart-btn"><i class="fas fa-shopping-cart"></i> Add
to Cart</a>
        </div>
      </div>
      <div class="col-lg-4 col-md-6 text-center berry">
        <div class="single-product-item">
          <div class="product-image">
            <a href="/guestmaster/singleproduct"></a>
          </div>
          <h3>Apple iPhone 13 (128GB,White)</h3>
          <p class="product-price"><span></span>Rs.44900</p>
          <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
        </div>
      </div>
      <div class="col-lg-4 col-md-6 text-center lemon">
        <div class="single-product-item">
          <div class="product-image">
            <a href="/guestmaster/singleproduct"></a>
          </div>
          <h3>Apple iPhone 15(128GB,Blue)</h3>
          <p class="product-price"><span></span> Rs.63700 </p>

```

```

        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
<div class="col-lg-4 col-md-6 text-center">
    <div class="single-product-item">
        <div class="product-image">
            <a href="/guestmaster/singleproduct"></a>
        </div>
        <h3>Apple iPhone14(128GB,Starlight)</h3>
        <p class="product-price"><span></span> Rs.51990</p>
        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
<div class="col-lg-4 col-md-6 text-center">
    <div class="single-product-item">
        <div class="product-image">
            <a href="/guestmaster/singleproduct"></a>
        </div>
        <h3>Apple iPhone 16e (512GB, Black)</h3>
        <p class="product-price"><span></span> Rs.83400 </p>
        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
<div class="col-lg-4 col-md-6 text-center strawberry">
    <div class="single-product-item">
        <div class="product-image">
            <a href="/guestmaster/singleproduct"></a>
        </div>
        <h3>Apple Watch Series 10 GPS </h3>
        <p class="product-price"><span></span>Rs.45490 </p>
        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
</div>
<div class="row">
    <div class="col-lg-12 text-center">
        <div class="pagination-wrap">
            <ul>
                <li><a href="#">Prev</a></li>
                <li><a href="#">1</a></li>
                <li><a class="active" href="#">2</a></li>
                <li><a href="#">3</a></li>

```

```

        <li><a href="#">Next</a></li>
    </ul>
</div>
</div>
</div>
</div>
</div>
</div>
<!-- end products -->

```

Singleproduct.component.ts

```

import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";

@Injectable({
  providedIn: "root",
})
export class ProductService {
  private apiUrl = "http://localhost:3000/product";

  constructor(private http: HttpClient) {}

  getProduct(productId: number) {
    return this.http.get(`${this.apiUrl}/${productId}`);
  }
}

```

Singleproduct.js

```

app.get("/product/:id", (req, res) => {
  const productId = req.params.id;
  db.query("SELECT * FROM products WHERE id = ?", [productId], (err, result) => {
    if (err) throw err;
    res.json(result[0]); // Send product details as JSON
  });
});

```

Singleproduct.component.html

```

<!-- single product -->
<div class="single-product mt-150 mb-150">
  <div class="container">
    <div class="row">
      <div class="col-md-5">
        <div class="single-product-img">
          
        </div>
      </div>
      <div class="col-md-7">

```

```

<div class="single-product-content">
  <h3>Apple iPhone 13 (128GB) - Blue</h3>
  <p class="single-product-pricing"><span></span>Rs.43900</p>
  <p>Brand-Apple<br>
    Operating System   iOS 14<br>
    Memory Storage Capacity 128 GB<br>
    Screen Size 6.1 Inches<br>
    Resolution 4K</p>
  <div class="single-product-form">
    <form action="index.html">
      <input type="number" placeholder="1">
    </form>
    <a href="cart" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    <p><strong>Categories: </strong>Mobile</p>
  </div>
  <h4>Share:</h4>
  <ul class="product-share">
    <li><a href=""><i class="fab fa-facebook-f"></i></a></li>
    <li><a href=""><i class="fab fa-twitter"></i></a></li>
    <li><a href=""><i class="fab fa-google-plus-g"></i></a></li>
    <li><a href=""><i class="fab fa-linkedin"></i></a></li>
  </ul>
</div>
</div>
</div>
</div>
</div>
<!-- end single product -->

<!-- more products -->
<div class="more-products mb-150">
  <div class="container">
    <div class="row">
      <div class="col-lg-8 offset-lg-2 text-center">
        <div class="section-title">
          <h3><span class="orange-text">Frequently bought together</span> </h3>

        </div>
      </div>
    </div>
    <div class="row">
      <div class="col-lg-4 col-md-6 text-center">
        <div class="single-product-item">
          <div class="product-image">
            <a href="single-product.html"></a>
          </div>
          <h3>Apple iPhone 15(128GB,Blue) 63700</h3>
          <p class="product-price"><span>Per Kg</span> Rs.51990 </p>

```

```

        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
<div class="col-lg-4 col-md-6 text-center">
    <div class="single-product-item">
        <div class="product-image">
            <a href="single-product.html"></a>
        </div>
        <h3>Apple iPhone14(128GB,Starlight)</h3>
        <p class="product-price"><span></span> Rs.51990</p>
        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
<div class="col-lg-4 col-md-6 offset-lg-0 offset-md-3 text-center">
    <div class="single-product-item">
        <div class="product-image">
            <a href="single-product.html"></a>
        </div>
        <h3>Apple iPhone 16e (512GB, Black)</h3>
        <p class="product-price"><span></span> Rs.83400 </p>
        <a href="cart.html" class="cart-btn"><i class="fas fa-shopping-cart"></i>
Add to Cart</a>
    </div>
</div>
</div>
</div>
</div>
</div>
<!-- end more products -->

```

Cart.component.ts

```

import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";

@Injectable({
  providedIn: "root",
})
export class CartService {
  private apiUrl = "http://localhost:3000/cart";

  constructor(private http: HttpClient) {}

  getCart(userId: number) {
    return this.http.get(`${this.apiUrl}/${userId}`);
  }
}

```

```

addToCart(cartItem: any) {
  return this.http.post(this.apiUrl, cartItem);
}

removeFromCart(cartId: number) {
  return this.http.delete(`${this.apiUrl}/${cartId}`);
}
}

```

Cart.component.html

```

<!-- cart -->
<div class="cart-section mt-150 mb-150">
  <div class="container">
    <div class="row">
      <div class="col-lg-8 col-md-12">
        <div class="cart-table-wrap">
          <table class="cart-table">
            <thead class="cart-table-head">
              <tr class="table-head-row">
                <th class="product-remove"></th>
                <th class="product-image">Product Image</th>
                <th class="product-name">Name</th>
                <th class="product-price">Price</th>
                <th class="product-quantity">Quantity</th>
                <th class="product-total">Total</th>
              </tr>
            </thead>
            <tbody>
              <tr class="table-body-row">
                <td class="product-remove"><a href="#"><i class="far fa-window-
close"></i></a></td>
                <td class="product-image"></td>
                <td class="product-name">Apple iphone 15</td>
                <td class="product-price">63700</td>
                <td class="product-quantity"><input type="number"
placeholder="1"></td>
                <td class="product-total">63700</td>
              </tr>
              <tr class="table-body-row">
                <td class="product-remove"><a href="#"><i class="far fa-window-
close"></i></a></td>
                <td class="product-image"></td>

```



```

        <td class="product-name">Apple iphone 14</td>
        <td class="product-price">51990</td>
        <td class="product-quantity"><input type="number"
placeholder="1"></td>
        <td class="product-total">51990</td>
    </tr>
    <tr class="table-body-row">
        <td class="product-remove"><a href="#"><i class="far fa-window-
close"></i></a></td>
        <td class="product-image"></td>
        <td class="product-name">Apple iphone 16e</td>
        <td class="product-price">83400</td>
        <td class="product-quantity"><input type="number"
placeholder="1"></td>
        <td class="product-total">83400</td>
    </tr>
</tbody>
</table>
</div>
</div>

```

```

<div class="col-lg-4">
    <div class="total-section">
        <table class="total-table">
            <thead class="total-table-head">
                <tr class="table-total-row">
                    <th>Total</th>
                    <th>Price</th>
                </tr>
            </thead>
            <tbody>
                <tr class="total-data">
                    <td><strong>Subtotal: </strong></td>
                    <td>199090</td>
                </tr>
                <tr class="total-data">
                    <td><strong>Shipping: </strong></td>
                    <td>200</td>
                </tr>
                <tr class="total-data">
                    <td><strong>Total: </strong></td>
                    <td>199290</td>
                </tr>
            </tbody>
        </table>
    </div>

```

```

</table>
<div class="cart-buttons">
  <a href="cart" class="boxed-btn">Update Cart</a>
  <a href="checkout" class="boxed-btn black">Check Out</a>
</div>
</div>

<div class="coupon-section">
  <h3>Apply Coupon</h3>
  <div class="coupon-form-wrap">
    <form action="index.html">
      <p><input type="text" placeholder="Coupon"></p>
      <p><input type="submit" value="Apply"></p>
    </form>
  </div>
</div>
</div>
</div>
</div>
</div>
<!-- end cart -->

```

Checkout.component.ts

```

import { Component, OnInit } from "@angular/core";
import { CheckoutService } from "../services/checkout.service";

@Component({
  selector: "app-checkout",
  templateUrl: "../checkout.component.html",
})
export class CheckoutComponent implements OnInit {
  cartItems = []; // Load from cart service
  totalPrice = 0;

  constructor(private checkoutService: CheckoutService) {}

  ngOnInit() {
    // Fetch cart items and calculate total price
    this.cartItems = [
      { product_id: 1, name: "Product A", quantity: 2, price: 500 },
      { product_id: 2, name: "Product B", quantity: 1, price: 300 },
    ];
  }
}

```

```

];
this.totalPrice = this.cartItems.reduce((sum, item) => sum + item.quantity * item.price,
0);
}

checkout() {
  const orderData = {
    user_id: 1, // Example user ID
    items: this.cartItems,
    total_price: this.totalPrice,
  };

  this.checkoutService.placeOrder(orderData).subscribe((res) => {
    alert("Order placed successfully!");
  });
}
}

```

Checkout.component.html

```

<!-- check out section -->
<div class="checkout-section mt-150 mb-150">
  <div class="container">
    <div class="row">
      <div class="col-lg-8">
        <div class="checkout-accordion-wrap">
          <div class="accordion" id="accordionExample">
            <div class="card single-accordion">
              <div class="card-header" id="headingOne">
                <h5 class="mb-0">
                  <button class="btn btn-link" type="button" data-toggle="collapse"
data-target="#collapseOne" aria-expanded="true" aria-controls="collapseOne">
                    Billing Address
                  </button>
                </h5>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

<div id="collapseOne" class="collapse show" aria-
labelledby="headingOne" data-parent="#accordionExample">
  <div class="card-body">
    <div class="billing-address-form">
      <form action="index.html">
        <p><input type="text" placeholder="Name"></p>
        <p><input type="email" placeholder="Email"></p>
        <p><input type="text" placeholder="Address"></p>
        <p><input type="text" placeholder="City"></p>
        <p><input type="text" placeholder="State"></p>
        <p><input type="tel" placeholder="PIN"></p>
        <p><input type="tel" placeholder="Phone"></p>
        <!-- <p><textarea name="bill" id="bill" cols="30" rows="10"
placeholder="Say Something"></textarea></p> -->
      </form>
    </div>
  </div>
</div>
<div class="card single-accordion">
  <div class="card-header" id="headingTwo">
    <h5 class="mb-0">
      <button class="btn btn-link collapsed" type="button" data-
toggle="collapse" data-target="#collapseTwo" aria-expanded="false" aria-
controls="collapseTwo">
        Shipping Address
      </button>
    </h5>
  </div>
  <div id="collapseTwo" class="collapse" aria-labelledby="headingTwo"
data-parent="#accordionExample">
    <div class="card-body">
      <div class="shipping-address-form">
        <p>Your shipping address form is here.</p>

```

```

        </div>
    </div>
</div>
</div>
<div class="card single-accordion">
    <div class="card-header" id="headingThree">
        <h5 class="mb-0">
            <button class="btn btn-link collapsed" type="button" data-
toggle="collapse" data-target="#collapseThree" aria-expanded="false" aria-
controls="collapseThree">
                Card Details
            </button>
        </h5>
    </div>
    <div id="collapseThree" class="collapse" aria-
labelledby="headingThree" data-parent="#accordionExample">
        <div class="card-body">
            <div class="card-details">
                <p>Your card details goes here.</p>
            </div>
        </div>
    </div>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
<div class="col-lg-4">
    <div class="order-details-wrap">
        <table class="order-details">
            <thead>
                <tr>
                    <th>Your order Details</th>
                    <th>Price</th>

```

```
</tr>
</thead>
<tbody class="order-details-body">
  <tr>
    <td>Product</td>
    <td>Total</td>
  </tr>
  <tr>
    <td>Apple iPhone 15</td>
    <td>63700</td>
  </tr>
  <tr>
    <td>Apple iPhone14</td>
    <td>51990</td>
  </tr>
  <tr>
    <td>Apple iPhone 16e</td>
    <td>83400</td>
  </tr>
</tbody>
<tbody class="checkout-details">
  <tr>
    <td>Subtotal</td>
    <td>199090</td>
  </tr>
  <tr>
    <td>Shipping</td>
    <td>200</td>
  </tr>
  <tr>
    <td>Total</td>
    <td>199290</td>
```

```

        </tr>
      </tbody>
    </table>
    <a href="#" class="boxed-btn">Place Order</a>
  </div>
</div>
</div>
</div>
</div>
<!-- end check out section -->

```

Addcategory.component.ts

```

import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";

@Injectable({
  providedIn: "root",
})
export class CategoryService {
  private apiUrl = "http://localhost:3000/categories";

  constructor(private http: HttpClient) {}

  addCategory(categoryData: any) {
    return this.http.post(this.apiUrl, categoryData);
  }
}

```

Addcategory.component.js

```

const express = require("express");
const mysql = require("mysql");
const cors = require("cors");

const app = express();
app.use(express.json());
app.use(cors());

const db = mysql.createConnection({

```

```

    host: "localhost",
    user: "root",
    password: "",
    database: "ecommerce_db",
  });

  db.connect((err) => {
    if (err) throw err;
    console.log("Connected to MySQL");
  });

  app.post("/categories", (req, res) => {
    const { name, description } = req.body;
    db.query("INSERT INTO categories (name, description) VALUES (?, ?)", [name,
    description], (err) => {
      if (err) return res.status(500).json({ error: err.message });
      res.json({ message: "Category added successfully!" });
    });
  });

  app.listen(3000, () => console.log("Server running on port 3000"));

```

addcatrgory.component.html

```

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Responsive Bootstrap4 Shop Template, Created by
  Imran Hossain from https://imransdesign.com/">
  <!-- title -->
  <title>Mobmetrics</title>
  <!-- favicon -->
  <link rel="shortcut icon" type="image/png"
  href="/assets/guest/assets/img/favicon.png">
  <!-- google font -->
  <link href="https://fonts.googleapis.com/css?family=Open+Sans:300,400,700"
  rel="stylesheet">
  <link href="https://fonts.googleapis.com/css?family=Poppins:400,700&display=swap"
  rel="stylesheet">
  <!-- fontawesome -->
  <link rel="stylesheet" href="/assets/guest/assets/css/all.min.css">
  <!-- bootstrap -->
  <link rel="stylesheet" href="/assets/guest/assets/bootstrap/css/bootstrap.min.css">
  <!-- owl carousel -->
  <link rel="stylesheet" href="/assets/guest/assets/css/owl.carousel.css">

```



```

<!-- magnific popup -->
<link rel="stylesheet" href="/assets/guest/assets/css/magnific-popup.css">
<!-- animate css -->
<link rel="stylesheet" href="/assets/guest/assets/css/animate.css">
<!-- mean menu css -->
<link rel="stylesheet" href="/assets/guest/assets/css/meanmenu.min.css">
<!-- main style -->
<link rel="stylesheet" href="/assets/guest/assets/css/main.css">
<!-- responsive -->
<link rel="stylesheet" href="/assets/guest/assets/css/responsive.css">
</head>

<!-- header -->
<div class="top-header-area" id="sticker">
  <div class="container">
    <div class="row">
      <div class="col-lg-12 col-sm-12 text-center">
        <div class="main-menu-wrap">
          <!-- logo -->
          <div class="site-logo">
            <a href="/guestmaster/guestindex">
              
            </a>
          </div>
          <!-- logo -->
        <!-- menu start -->
        <nav class="main-menu">
          <ul>
            <li class="current-list-item"><a
href="/guestmaster/guestindex">Home</a>
            </li>
            <li><a href="about.html">About</a></li>
            <li><a href="#">Pages</a>
            <ul class="sub-menu">
              <li><a href="404.html">404 page</a></li>
              <li><a href="about.html">About</a></li>
              <li><a href="cart.html">Cart</a></li>
              <li><a href="checkout.html">Check Out</a></li>
              <li><a href="contact.html">Contact</a></li>
              <li><a href="news.html">News</a></li>
              <li><a href="shop.html">Shop</a></li>
            </ul>
            </li>
            <li><a href="news.html">News</a>
            <ul class="sub-menu">
              <li><a href="news.html">News</a></li>
              <li><a href="single-news.html">Single News</a></li>
            </ul>
            </li>
          </ul>
        </nav>
      </div>
    </div>
  </div>

```

```

        <li><a href="contact.html">Contact</a></li>
        <li><a href="shop.html">Shop</a>
            <ul class="sub-menu">
                <li><a href="shop.html">Shop</a></li>
                <li><a href="checkout.html">Check Out</a></li>
                <li><a href="single-product.html">Single Product</a></li>
                <li><a href="cart.html">Cart</a></li>
            </ul>
        </li>
        <li><a href="login">Login</a></li>
        <li>
            <div class="header-icons">
                <a class="shopping-cart" href="cart.html"><i class="fas fa-
shopping-cart"></i></a>
                <a class="mobile-hide search-bar-icon" href="#"><i class="fas
fa-search"></i></a>
            </div>
        </li>
    </ul>
</nav>
    <a class="mobile-show search-bar-icon" href="#"><i class="fas fa-
search"></i></a>
    <div class="mobile-menu"></div>
    <!-- menu end -->
</div>
</div>
</div>
</div>
</div>
<!-- end header -->

<!-- search area -->
<div class="search-area">
    <div class="container">
        <div class="row">
            <div class="col-lg-12">
                <span class="close-btn"><i class="fas fa-window-close"></i></span>
                <div class="search-bar">
                    <div class="search-bar-tablecell">
                        <h3>Search For:</h3>
                        <input type="text" placeholder="Keywords">
                        <button type="submit">Search <i class="fas fa-search"></i></button>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
<!-- end search area -->

```

```

<form (submit)="addCategory()">
  <input [(ngModel)]="category.name" placeholder="Category Name" required />
  <textarea [(ngModel)]="category.description" placeholder="Category
Description"></textarea>
  <button type="submit">Add Category</button>
</form>

<!-- footer -->
<div class="footer-area">
  <div class="container">
    <div class="row">
      <div class="col-lg-3 col-md-6">
        <div class="footer-box about-widget">
          <h2 class="widget-title">About us</h2>
          <p>Start selling with us and reach millions of our customers,Place your
products in front of high-intent shoppers and convert them with ease</p>
        </div>
      </div>
      <div class="col-lg-3 col-md-6">
        <div class="footer-box get-in-touch">
          <h2 class="widget-title">Get in Touch</h2>
          <ul>
            <li>34/8, East Hukupara, Gifirtok, Sadan.</li>
            <li></li>
            <li>+00 111 222 3333</li>
          </ul>
        </div>
      </div>
      <div class="col-lg-3 col-md-6">
        <div class="footer-box pages">
          <h2 class="widget-title">Pages</h2>
          <ul>
            <li><a href="index.html">Home</a></li>
            <li><a href="about.html">About</a></li>
            <li><a href="services.html">Shop</a></li>
            <li><a href="news.html">News</a></li>
            <li><a href="contact.html">Contact</a></li>
          </ul>
        </div>
      </div>
      <div class="col-lg-3 col-md-6">
        <div class="footer-box subscribe">
          <h2 class="widget-title">Subscribe</h2>
          <p>Subscribe to our mailing list to get the latest updates.</p>
          <form action="index.html">
            <input type="email" placeholder="Email">
            <button type="submit"><i class="fas fa-paper-plane"></i></button>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

    </div>
</div>
<!-- end footer -->

<!-- copyright -->
<div class="copyright">
    <div class="container">
        <div class="row">
            <div class="col-lg-6 col-md-12">

                </div>
                <div class="col-lg-6 text-right col-md-12">
                    <div class="social-icons">
                        <ul>
                            <li><a href="#" target="_blank"><i class="fab fa-facebook-
f"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
twitter"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
instagram"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
linkedin"></i></a></li>
                            <li><a href="#" target="_blank"><i class="fab fa-
dribbble"></i></a></li>
                        </ul>
                    </div>
                </div>
            </div>
        </div>
    </div>
<!-- end copyright -->

<!-- jquery -->
<script src="/assets/guest/assets/js/jquery-1.11.3.min.js"></script>
<!-- bootstrap -->
<script src="/assets/guest/assets/bootstrap/js/bootstrap.min.js"></script>
<!-- count down -->
<script src="/assets/guest/assets/js/jquery.countdown.js"></script>
<!-- isotope -->
<script src="/assets/guest/assets/js/jquery.isotope-3.0.6.min.js"></script>
<!-- waypoints -->
<script src="/assets/guest/assets/js/waypoints.js"></script>
<!-- owl carousel -->
<script src="/assets/guest/assets/js/owl.carousel.min.js"></script>
<!-- magnific popup -->
<script src="/assets/guest/assets/js/jquery.magnific-popup.min.js"></script>
<!-- mean menu -->
<script src="/assets/guest/assets/js/jquery.meanmenu.min.js"></script>
<!-- sticker js -->
<script src="/assets/guest/assets/js/sticker.js"></script>

```

```
<!-- main js -->
<script src="/assets/guest/assets/js/main.js"></script>
```

Addproduct.component.ts

```
import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";

@Injectable({
  providedIn: "root",
})
export class ProductService {
  private apiUrl = "http://localhost:3000/products";

  constructor(private http: HttpClient) {}

  addProduct(productData: any) {
    return this.http.post(this.apiUrl, productData);
  }
}
import { Component } from "@angular/core";
import { ProductService } from "../services/product.service";

@Component({
  selector: "app-add-product",
  templateUrl: "./add-product.component.html",
})
export class AddProductComponent {
  product = { name: "", description: "", price: "", image: "", category: "" };

  constructor(private productService: ProductService) {}

  addProduct() {
    this.productService.addProduct(this.product).subscribe((res) => {
      alert("Product added successfully!");
    });
  }
}
```

Addproduct.component.html

```
<form (submit)="addProduct()">
  <input [(ngModel)]="product.name" placeholder="Product Name" required />
  <textarea [(ngModel)]="product.description" placeholder="Product Description"
required></textarea>
  <input [(ngModel)]="product.price" type="number" placeholder="Price" required />
  <input [(ngModel)]="product.image" placeholder="Image URL" required />
```

```
<select [(ngModel)]="product.category" required>
  <option value="Mobile">Mobile</option>
  <option value="Electronics">Smartwatches</option>
  <option value="Electronics">Accessories</option>
</select>
<button type="submit">Add Product</button>
</form>
```

6. TESTING

Software testing is a critical element of software assurance and represents the ultimate review of specification, design and coding. Testing includes test case design and their execution. Testing demonstrates the software functions according to the specification.

Many types of testing are done in various stages of development to detect errors in the software and corrected. In testing the internal logic of the program, the inputs and the outputs are tested. Each test has different purpose, all works to verify that the system elements have been properly integrated and perform allocated functions.

TESTING OBJECTIVES

- Testing is the process of executing a program with intension of an error
- A good test case is one that has high probability of finding a yet undiscovered error
- Purpose of testing
- To affirm the quality of a product
- To find and eliminate any residual error
- To demonstrate the presence of all specified functions in the product
- To validate the software as a solution to the original problem. In our system we
- performed the following testing:

TESTING STRATEGY

6.1. Unit Testing

The modules are tested separately. This is done during programming stage itself. Any logical errors found are corrected. Finally ensure that each module is working as expected. In our system there are four modules which we have successfully tested separately.

Data Validation Testing

After recovering all logical and interface errors, the software is completely assembled as a package. Then perform validation testing by inputting dummy data and ensure that it satisfies all the requirements of the user. It helps performance requirements and us to ensure that the software meets all functional behaviours.

System Testing

The chapter testing describes various testing methodologies which are adapted and detailed view of test data within each database. During testing of a program to be tested is executed with a set of test data and the output of the program for test data is evaluated.

In a software development project, errors can be introduced at any stage during development. The errors are detected after each phase by techniques like inspections, but some errors remain undetected. Ultimately, these remaining errors will be reflected in the code. Hence the final code is likely to have some requirement errors or design errors, in addition to errors introduced during the coding activity. Testing is the activity where the errors remaining from all the previous phases must be detected. During testing, the software to be tested is executed with a set of test cases, and the behaviour of the system

TEST CASES

Test cases are a set of instructions on how to validate a particular test objective or target, which when followed will tell us if the expected behaviour of the system is satisfied or not.

A test case has components that describe input, action and response, to determine if a feature of the application is working correctly.

6.2 Integration Testing

After unit testing all modules are integrated and flow of information between the modules has been tested. This testing is done with sample data. There by find out the overall performance of the system. In our case study we have dependent modules which we have performed integration testing successfully and ensured the flow of information between modules as expected.

6.2.1 User authentication integration test

Test Name	User Registration and Login Integration
Components Involved	User Registration Module, User Authentication Module, Database
Test Description	Verify that a new user can register and subsequently login with their credentials
Test Steps	1. Navigate to registration page
	2. Enter valid user details
	3. Submit registration form
	4. Logout
	5. Navigate to login page
	6. Enter registered credentials
	7. Submit login form
Expected Result	User should be able to register successfully and then login with the registered credentials
Actual Result	User registration successful. Login authentication works correctly with registered credentials. User session is created properly.
Status	PASS
Issues Found	None

6.2.2 Product listing and search integration

Test Name	Product Listing and Search Integration
Components Involved	product Database, Search Module, Front-end Display
Test Description	Verify that product listing and search functionality work correctly together
Test Steps	1. Navigate to homepage
	2. Verify current products are displayed
	3. Use search function to find a specific product
	4. Click on product to view details
Expected Result	Homepage should display available product from database. Search should return relevant results. Clicking a product should show its details.
Actual Result	Product are correctly retrieved from database and displayed. Search function returns accurate results. product details are displayed correctly when a product is selected.
Status	Pass

6.2.3 Product selection and ordering

Test Name	product Selection and ordering
Components Involved	Product selection Module, cart module,
Test Description	Verify that users can select product and make an order
Test Steps	1. Select a product
	2. select number of item
	4. View price and details
	5. add to cart
	6. Proceed to payment
Expected Result	User should be able to view product, select them, and move to payment. Selected product should be temporarily locked during the ordering process.
Actual Result	product selection works correctly. Database is updated correctly after the process

6.2.4 Admin panel and database integration

Test Name	Admin Panel and Database Integration
Components Involved	Admin Module, Product Database, User Database
Test Description	Verify that admin can add/update categories and manage customer accounts and seller account
Test Steps	1. Login as admin
	2. Add category,products
	3. verify seller
	4. View user accounts
	5. Verify changes in frontend
Expected Result	Admin should be able to manage products and user(customer and seller) accounts. Changes should be reflected in the database and frontend.
Actual Result	products addition and updates are successfully saved to database. User management functions work correctly.

6.3 Code Improvement

6.3.1 Refactoring:

- Improving code structure without changing functionality.
- Enhancing readability, maintainability, and efficiency.
- Removing code duplication.

6.3.2 Optimization:

- Improving the performance of the code.
- Reducing execution time and resource consumption.

6.3.3 Code Reviews:

- Having project guide review the code.
- Identifying potential issues and areas for improvement.
- Ensuring code quality and consistency.

6.3.4 Version Control:

- Using Git to track code changes.
- Enabling easy rollback to previous versions if needed.

7.INPUT OUTPUT SCREEN

The image displays two screenshots of the Mobmetrics E-commerce website. The top screenshot shows the homepage with a dark background featuring various mobile accessories like phone stands and cases. The text 'MOB METRICS' is at the top, followed by 'Mobiles and Electronics' in large white letters. Navigation links include Home, About, Pages, News, Contact, Shop, and Login. There are also buttons for 'Products' and 'Contact Us'. The bottom screenshot shows a browser window with the URL 'localhost:4200/login'. The login form is centered on a light gray background. It has a title 'Login', a username field with 'admin', a password field with masked characters and an eye icon, a 'Login' button, and a link 'Don't have an account? Sign Up'.

localhost:4200/guestmaster/guestindex

MOB METRICS

Home About Pages News Contact Shop Login

MOB METRICS

Mobiles and Electronics

Products Contact Us

Mobmetrics

localhost:4200/login

Login

admin

Login

Don't have an account? Sign Up

Sign Up

Please fill in this form to create an account.

First Name

Last Name

Address

PIN

Phone Number

Username

Email

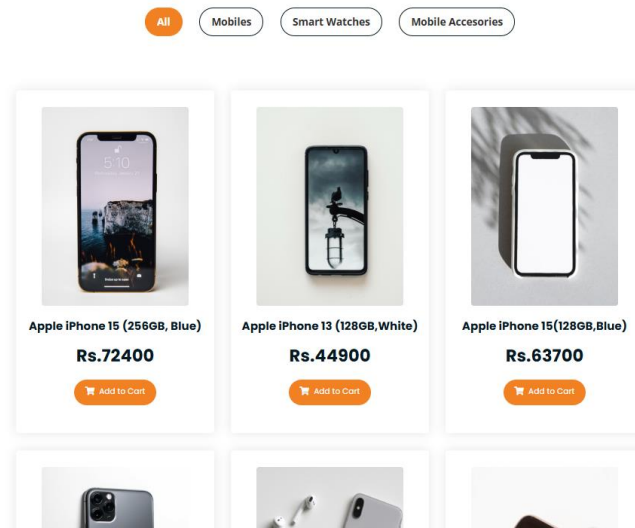
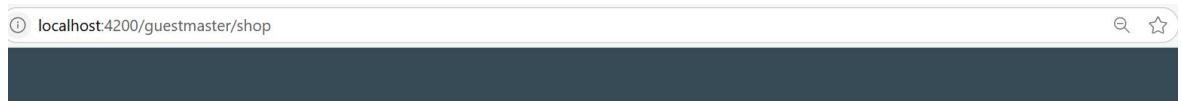
Password

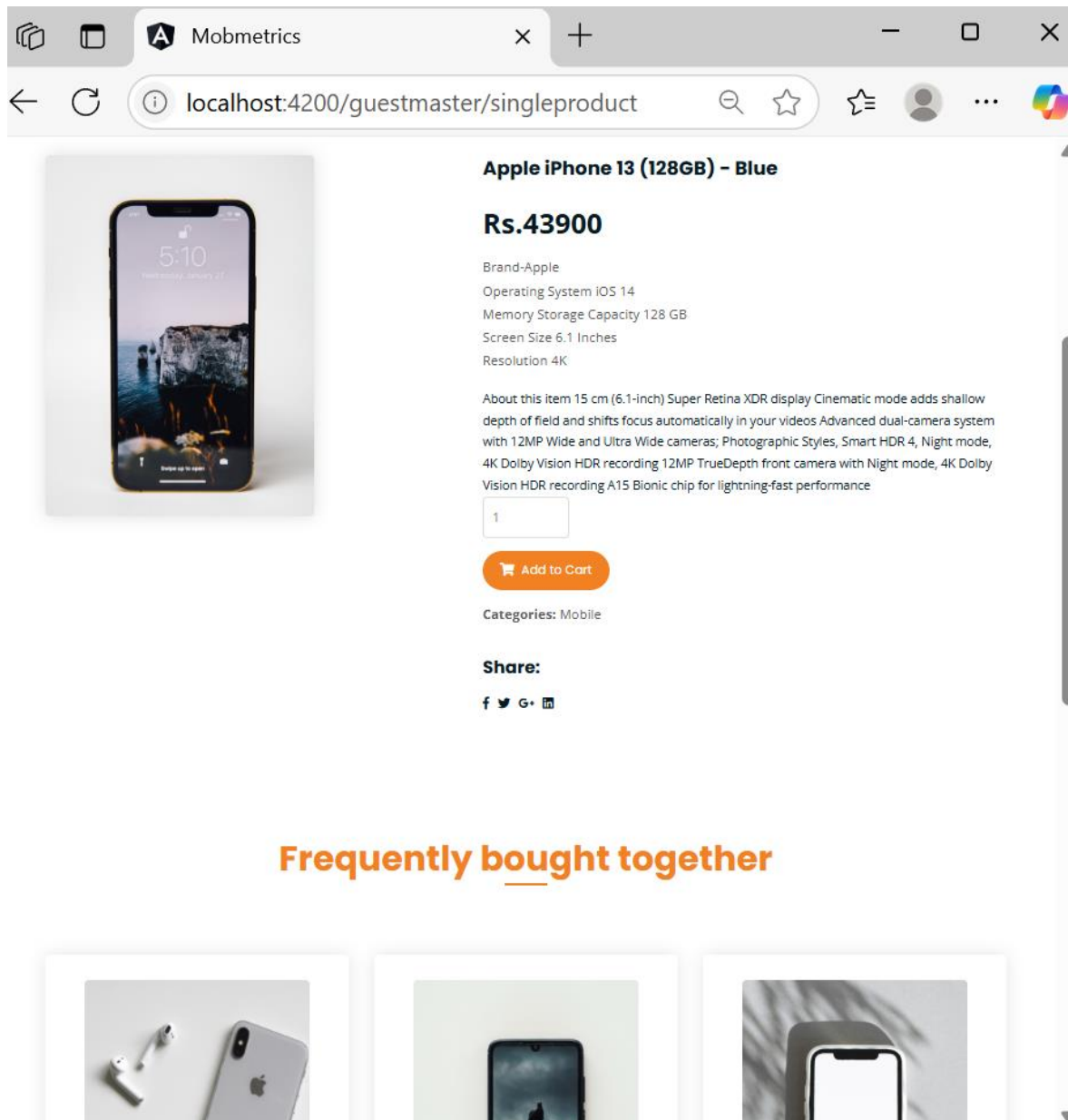
Confirm Password

By creating an account you agree to our [Terms & Privacy](#).

Sign Up

[Watch on YouTube](#)





Apple iPhone 13 (128GB) - Blue

Rs.43900

Brand-Apple
Operating System iOS 14
Memory Storage Capacity 128 GB
Screen Size 6.1 Inches
Resolution 4K

About this item 15 cm (6.1-inch) Super Retina XDR display Cinematic mode adds shallow depth of field and shifts focus automatically in your videos Advanced dual-camera system with 12MP Wide and Ultra Wide cameras; Photographic Styles, Smart HDR 4, Night mode, 4K Dolby Vision HDR recording 12MP TrueDepth front camera with Night mode, 4K Dolby Vision HDR recording A15 Bionic chip for lightning-fast performance

1

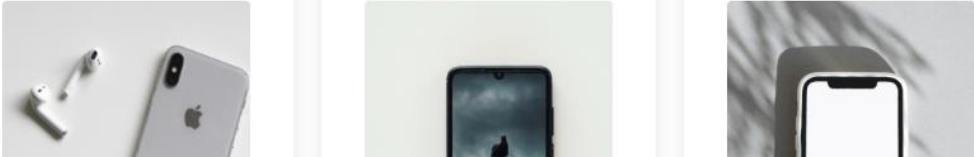
Add to Cart

Categories: Mobile

Share:

f t G+ in

Frequently bought together



Browser: Mobmetrics | URL: localhost:4200/cart

Navigation: Home | About | Pages | News | Contact | Shop | Login

MOBMETRICS Cart

	Product Image	Name	Price	Quantity	Total
☐		Apple iphone 15	63700	1	63700
☐		Apple iphone 14	51990	1	51990
☐		Apple iphone 16e	83400	1	83400

Total	Price
Subtotal:	199090
Shipping:	200
Total:	199290

[Update Cart](#) [Check Out](#)

Apply Coupon

[APPLY](#)

Billing Address

Name

Email

Address

City

State

PIN

Phone

Product	Price
Apple iPhone 15	63700
Apple iPhone14	51990
Apple iPhone 16e	83400
Subtotal	199090
Shipping	200
Total	199290

Place Order

Admin

Add Category

Category Name

Smart Watches

Description

Smart watches and wearables

Submit

8. IMPLEMENTATION PLAN

Implementation is a process of converting a new system into an operational one. The designed system is converted into an operational one using a suitable programming language.

Implementation includes all these activities that take place to convert an old system into a new one. Proper implementation is necessary to provide a reliable system to meet organizational requirements.

Implementation Details

In the implementation phase all the programs are written, a database is created, user operational documents are written, users are trained, and the system tested with operational data. The implementation is carried out with the results that have been obtained from the feasibility study and analysis. The system is implemented by finishing the project with the help of appropriate tools. Then the system is tested with appropriate data inputs to check the successfulness of the system. This is being carried out by inputting data that is rare to be inputted. Then the administrator being trained of the operational functionalities to control and maintain system at later stage. The third-party users' role is being carried out by the implementation team itself.

There by it is made sure that the system meets the required standards. Implementation is the final and important phase. The system can be implemented only after through testing is done and it is found to work according to the specification. This method also offers the greatest security since the old system can take over if the errors are found or inability to handle certain type of transactions while using the new system. The implementation plan includes a description of all activities that must occur to implement the system and to put it into operation. It indicates the personal responsible for the activities and prepares a time chart for implementing the system.

Implementation Plan

Implementation includes all those activities that take place to convert from the old system to the new one. The new system may be totally new, replacing an existing manual or automated system. Proper implementation is essential to provide a reliable system to meet customer requirements. The process of putting developed system in actual use is called system implementation.

9.LIMITATION OF PROJECT

- **Data Accuracy Dependency:** The system's accuracy is inherently dependent on the accuracy of the input provided by users and administrators. If users enter incorrect information (e.g., personal details, order information), or if administrators make errors in updating product stock or price, the system will reflect those inaccuracies. Therefore, while the system itself strives for accuracy, it cannot guarantee perfect data integrity if the input is flawed.
- **Potential for Bugs:** Like any software, the e-commerce system is susceptible to bugs or errors. The documents specifically mention two potential sources of issues:
 - **Payment Gateway Lapses:** Problems with the payment gateway, which is a third-party service, can disrupt transactions. These lapses might include processing delays, transaction failures, or security vulnerabilities in the gateway itself.
 - **Unstable Connectivity:** The system's reliance on internet connectivity means that unstable connections on the user's side can lead to problems. These issues may include interruptions during payment, incomplete transactions, or the inability to access the system.

10. FUTURE APPLICATIONS OF THE PROJECT

- **Mobile Application Development:** Recognizing the increasing prevalence of mobile devices, developing dedicated mobile applications for iOS and Android platforms is proposed. This would offer users a more convenient and accessible way to browse product, make order, and manage their order on the go. Mobile apps can also leverage device-specific features to enhance the user experience
- **Push Notifications:** Mobile apps can utilize push notifications to provide users with timely updates and alerts, such as reminders for newly launched products, or special promotions.
- **Multi-Language Support:** To broaden the system's reach and cater to a more diverse user base, the implementation of multi-language support is suggested. This would involve translating the user interface, content, and system messages into multiple languages, making the platform accessible to users who prefer or require languages other than English.
- **Social Media Integration:** Integrating the system with social media platforms can enhance user engagement and promote product sharing. This feature would allow users to share their product preferences with their friends and followers directly from the application, fostering a social aspect to tech enthusiast.
- **Expansion and Performance:**
 - The system has the potential to expand its partnerships with more sellers increasing the number of products and type of categories available to users.
 - Improvements can be made to increase database access speed, leading to faster response times and a better user experience.

11.CONCLUSION

In summary, MOBMETRICS E-COMMERCE WEBSITE is designed to revolutionize the way mobiles and electronics items are purchased and managed, offering a more accessible, efficient, and user-friendly experience for both customers and sellers. By providing a platform for online shopping, the system eliminates the traditional inconveniences associated with physical purchases, such as traveling to the market and waiting in long queues. This not only saves time and effort for customers but also empowers them with the ability to order product from any location at any time.

Furthermore, the system streamlines operations for staff by automating key processes, including product sales, and transaction management. This automation reduces the potential for errors, increases efficiency, and allows staff to focus on other essential tasks.

Ultimately, the mobmetrics e-commerce website aims to enhance the overall shopping experience by prioritizing convenience, efficiency, and user satisfaction.

12. BIBLIOGRAPHY

Freeman, A. (2018). Pro Angular. Apress.

- Covers Angular development best practices and component-based architecture.

Raj, R. (2021). Node.js Design Patterns. Packt Publishing.

- Explores Node.js design principles, RESTful API architecture, and scalability strategies.

Williams, J. (2020). SQL for Beginners. O'Reilly Media.

- Discusses MySQL database design, indexing, and optimization.

W3Schools. (n.d.). *Angular CRUD Example with Node.js & MySQL*.

- Retrieved from <https://www.w3schools.com>

FreeCodeCamp. (2022). Building an eCommerce Backend with Node.js.

- Retrieved from <https://www.freecodecamp.org>

Angular Documentation – <https://angular.io/docs>

Node.js Documentation – <https://nodejs.org/en/docs/>

MySQL Documentation – <https://dev.mysql.com/doc/>